

2η Εργασία: Νευρωνικά Δίκτυα για Ταξινόμηση Εικόνων

Τύπος εργασίας: Ατομική

Ημερομηνία παράδοσης: Τετάρτη 17/06/2026, 23:55 (Δεν θα δοθεί παράταση)

Τρόπος παράδοσης: Αποκλειστικά μέσω του eclass (εργασίες μέσω email δεν γίνονται δεκτές)

Σύνολο βαθμών: 100 (40 μονάδες Notebook + 60 μονάδες προφορική εξέταση)
(20% του τελικού βαθμού του μαθήματος)

Η προφορική εξέταση θα πραγματοποιηθεί την Δευτέρα 29/06/2026.

Στόχος

Σε αυτή την εργασία θα εκπαιδεύσετε νευρωνικά δίκτυα, Feedforward και Convolutional, για τη μορφολογική ταξινόμηση εικόνων ζωγραφικής σε τέσσερα καλλιτεχνικά κινήματα, χρησιμοποιώντας υποσύνολο του WikiArt. Θα αναπτύξετε ολόκληρο τον κύκλο ενός deep learning συστήματος σε PyTorch που περιλαμβάνει προετοιμασία δεδομένων, σχεδίαση αρχιτεκτονικών, training loop, model selection μέσω validation set, βελτιστοποίηση με σύγχρονες τεχνικές (BatchNorm, regularization, optimizers, data augmentation, LR scheduling), transfer learning από προεκπαιδευμένο ResNet18, και οπτικοποίηση των εσωτερικών αναπαραστάσεων.

Το notebook αποτελεί την προετοιμασία σας· και αντιστοιχεί στο 40% της τελικής βαθμολογίας. Η κύρια αξιολόγηση γίνεται στην προφορική εξέταση (60%), όπου θα ερωτηθείτε πάνω στον κώδικα και τα αποτελέσματά σας όπως έχουν υποβληθεί στο eclass. Αν τα αποτελέσματα (μοντέλα, διαγράμματα, metrics) δεν είναι διαθέσιμα ή δεν μπορείτε να εξηγήσετε τις επιλογές και τα αποτελέσματά σας, η βαθμολογία της εργασίας θα είναι αντίστοιχα χαμηλή, ανεξάρτητα από την ποιότητα του κώδικα.

Πλαίσιο

Η ταξινόμηση πινάκων σε καλλιτεχνικά κινήματα ήταν επί δεκαετίες αποκλειστικό έργο ειδικών. Σήμερα τα μουσεία διατηρούν εκατομμύρια ψηφιοποιημένα έργα και η αυτόματη υφολογική ταξινόμηση τέχνης είναι ενεργός τομέας στη διασταύρωση της Όρασης Υπολογιστών με τις Ψηφιακές Ανθρωπιστικές Σπουδές. Το ερώτημα δεν είναι μόνο αν τα νευρωνικά δίκτυα μπορούν να ταξινομήσουν πίνακες, αλλά και τι ακριβώς «βλέπουν» όταν το κάνουν.

Θα δουλέψετε με τέσσερα κινήματα, καθένα με διακριτή οπτική ταυτότητα:

- Μπαρόκ (1600–1750): δραματικός φωτισμός, σκούρο φόντο, ρεαλιστική απόδοση. Ζωγράφοι όπως οι: Caravaggio, Rembrandt, Vermeer.
- Ιμπρεσιονισμός (1860–1900): παστέλ παλέτα, ορατές χαλαρές πινελιές, υπαίθριες σκηνές. Ζωγράφοι όπως οι: Monet, Renoir, Pissarro.
- Κυβισμός (1907–1920): γεωμετρική θραυσματοποίηση, παλέτα σε ώχρες και γκρι, πολλαπλές οπτικές γωνίες σε ένα έργο. Ζωγράφοι όπως οι: Picasso, Braque.

- Αφαιρετικός Εξπρεσιονισμός (1940–1960): μη παραστατικά έργα, χειρονομιακές πιναελιές ή πεδία χρώματος, απουσία αναγνωρίσιμου θέματος. Ζωγράφοι όπως οι: Pollock, Rothko, de Kooning.

Δομή απαντήσεων

Η εργασία αποτελείται από 10 ερωτήματα (Q0.1–Q5.1) που απαντώνται εντός του notebook, σε κελί markdown αμέσως μετά τον αντίστοιχο κώδικα, με τίτλο ακριβώς ίδιο με αυτόν της εκφώνησης (π.χ. `## Q1.1 Με βάση τα loss curves...`). Κάθε απάντηση πρέπει να είναι σύντομη (5–10 προτάσεις), να απαντάει άμεσα στο ερώτημα, και να τεκμηριώνεται με αναφορά σε συγκεκριμένα αποτελέσματα του notebook (metrics, plots, πίνακες).

Απάντηση που παραμένει σε γενικό θεωρητικό επίπεδο, χωρίς αναφορά στα αποτελέσματα του notebook σας, θεωρείται ελλιπής.

0. Δεδομένα

0.1 Λήψη και προετοιμασία

Το WikiArt είναι το πιο διαδεδομένο dataset για υπολογιστική ανάλυση τέχνης, με περίπου 80.000 ψηφιοποιημένα έργα σε 27 κινήματα. Το πλήρες dataset απαιτεί περίπου 30GB, οπότε για την παρούσα εργασία σας παρέχεται έτοιμο προεπεξεργασμένο υποσύνολο τεσσάρων κλάσεων σε χαμηλή ανάλυση.

Κατεβάστε το αρχείο `wikiart_hw2.npz.zip` από το eclass (Έγγραφα → Δεδομένα 2ης Εργασίας 2026) και αποσυμπίεστε το για να εξάγετε το `wikiart_hw2.npz` (~15MB). Είναι ένα συμπίεσμένο numpy archive που περιέχει:

- `x`: array σχήματος (5400, 32, 32, 3), τύπου `uint8`, με εικόνες RGB σε `channels-last` format και τιμές pixel στο διάστημα 0–255. Οι εικόνες έχουν ήδη υποστεί `center-crop` στη μικρότερη πλευρά και `resize` σε 32×32.
- `y`: array σχήματος (5400,), τύπου `int64`, με τα labels 0–3.
- `class_names`: array με τα ονόματα κλάσεων στη σειρά των labels.

Το dataset είναι ισορροπημένο (1.350 δείγματα ανά κλάση) και ήδη `shuffled` με `random_state=42`. Η αντιστοιχία labels με κινήματα είναι: 0 → Baroque, 1 → Impressionism, 2 → Cubism, 3 → Abstract_Expressionism.

- Φορτώστε το αρχείο και μετατρέψτε τα δεδομένα σε PyTorch tensors. Τα `x` ως `float32`, κανονικοποιημένα στο διάστημα [0, 1] (διαίρεση με 255), και σε `channels-first` format [N, 3, 32, 32] (hint: `torch.permute`). Τα `y` ως `int64`.
- Δημιουργήστε ισορροπημένα splits με `train_test_split` (από `sklearn.model_selection`) χρησιμοποιώντας `stratify=y` και `random_state=42`. Στόχος: 800 train, 200 val, 350 test ανά κλάση (συνολικά 3200 / 800 / 1400). Επειδή η `train_test_split` παράγει δύο σύνολα κάθε φορά, καλέστε τη δύο φορές διαδοχικά.
- Δημιουργήστε PyTorch DataLoaders με `batch_size=16`, `shuffle=True` για train/val και `shuffle=False` για test.

- Εκτυπώστε στατιστικά splits και επιβεβαιώστε ότι κάθε σύνολο περιέχει τον αναμενόμενο αριθμό δειγμάτων ανά κλάση.
- Οπτικοποιήστε δύο τυχαίες εικόνες ανά κλάση σε σχήμα 2x4 (γραμμές: παραδείγματα, στήλες: κλάσεις).

Q0.1 (3 μον.)

Από την οπτικοποίηση, ποια οπτικά γνωρίσματα φαίνεται να διαφοροποιούν περισσότερο τα τέσσερα κινήματα μεταξύ τους (χρωματική παλέτα, υφή, σύνθεση, παρουσία θέματος); Ποια ζευγάρια κλάσεων αναμένετε να είναι πιο εύκολο να μπερδέψει ένας ταξινομητής, και γιατί; Αν ένας άνθρωπος έβλεπε τις ίδιες εικόνες resized σε 32x32, ποιο κίνημα θα ήταν πιο δύσκολο να ξεχωρίσει;

1. Feedforward Neural Network σε raw pixels

Πριν χρησιμοποιήσουμε μια αρχιτεκτονική σχεδιασμένη για εικόνες, ας δούμε τι κάνει ένα απλό FFN όταν παίρνει την εικόνα ως flat διάνυσμα 3.072 τιμών (32x32x3 flattened). Είναι το πρώτο σας PyTorch δίκτυο και θα χρησιμεύσει ως baseline για το CNN του Ερωτήματος 2.

1.1 Αρχιτεκτονική και εκπαίδευση

Ορίστε ως υποκλάση του `nn.Module` ένα FFN με είσοδο 3.072 (η εικόνα flattened), τρία πλήρως συνδεδεμένα επίπεδα με αριθμούς νευρώνων 128, 32, και 4, και ReLU activation μεταξύ των επιπέδων (αλλά όχι μετά την τελευταία γραμμική, που οδηγεί στο output).

Στη συνέχεια ορίστε δύο συναρτήσεις. Η `train_one_epoch` εκτελεί μία εποχή εκπαίδευσης (forward pass, υπολογισμός loss, backward pass, ενημέρωση παραμέτρων) και επιστρέφει το μέσο train loss της εποχής. Η `evaluate` περνά όλα τα batches ενός dataloader από το μοντέλο σε eval mode χωρίς να ενημερώνει τα βάρη, και επιστρέφει loss, accuracy, F1 macro averaged, και confusion matrix.

Εκπαιδεύστε το δίκτυο με optimizer Adam (lr=0.001), loss function `nn.CrossEntropyLoss`, και 30 εποχές. Σε κάθε εποχή αξιολογήστε στο validation set και κρατήστε το `state_dict` του μοντέλου με την καλύτερη επίδοση F1 macro στο validation. Στο τέλος φορτώστε αυτό το `state_dict` και αξιολογήστε στο test set.

Εκπαιδεύστε το δίκτυο τόσο σε CPU όσο και σε GPU (με ακριβώς τις ίδιες υπερπαραμέτρους και αριθμό εποχών), και τυπώστε τους χρόνους εκτέλεσης. Παρουσιάστε δύο διαγράμματα: (α) train και validation loss συναρτήσει της εποχής στο ίδιο σχήμα, (β) validation F1 macro συναρτήσει της εποχής με κάθετη γραμμή που σημαδεύει την εποχή επιλογής του best model (Για best model selection χρησιμοποιείται το validation F1 macro και όχι accuracy).

Q1.1 (5 μον.)

Με βάση τα loss curves και την καμπύλη validation F1, παρατηρείτε overfitting, underfitting, ή ισορροπία; Σε ποια εποχή επιλέχθηκε το best model με κριτήριο το validation F1, και σε ποια θα είχε επιλεγεί αν χρησιμοποιούσατε το train loss ως κριτήριο;

Γιατί η επιλογή με βάση το validation set είναι κρίσιμη; Ήταν αναμενόμενο το GPU speedup σε σχέση με την εκπαίδευση σε CPU για ένα δίκτυο αυτού του μεγέθους;

Q1.2 (3 μον.)

Παρουσιάστε accuracy, F1 macro, και confusion matrix του best model στο test set. Ποια κινήματα μπερδεύονται περισσότερο μεταξύ τους; Αναδιαμορφώστε (reshape) τα βάρη του πρώτου γραμμικού επιπέδου από σχήμα [128, 3072] σε [128, 3, 32, 32], κανονικοποιήστε κάθε «φίλτρο» στο [0, 1] (min-max ανά φίλτρο), και οπτικοποιήστε 16 από αυτά ως RGB εικόνες σε διάταξη 4x4. Συζητήστε γιατί το flat-pixel FFN δυσκολεύεται παρά την παρουσία της ReLU: τι «βλέπει» κάθε νευρώνας του πρώτου επιπέδου, και ποιο είδος δομής της εικόνας δεν μπορεί να εκμεταλλευτεί αυτή η αρχιτεκτονική;

2. Convolutional Neural Network

Σε αυτή την ενότητα θα χρησιμοποιήσετε τις εικόνες ως tensors (όχι flattened) και θα δείτε σταδιακά την επίδραση κάθε συστατικού ενός CNN στην εκπαίδευση και την επίδοση.

2.1 Σταδιακή κατασκευή

Ορίστε ένα CNN με ακολουθία τεσσάρων συνελικτικών επιπέδων (kernel size 5) που υλοποιεί την ακολουθία καναλιών 3, 16, 32, 64, 128 (όπου το αρχικό 3 αντιστοιχεί στα RGB κανάλια εισόδου). Η έξοδος του τελευταίου συνελικτικού επιπέδου εισέρχεται σε ένα fully-connected head τεσσάρων επιπέδων με αριθμό νευρώνων x , 1024, 256, 32, 4 (όπου x η διάσταση εξόδου του flatten η οποία μπορεί να διαφέρει μεταξύ των παρακάτω εκδόσεων).

Θα εκπαιδεύσετε τρεις εκδόσεις του δικτύου, καθεμία προσθέτοντας ένα συστατικό:

- §2.1 baseline: χωρίς padding, χωρίς pooling, χωρίς activations.
- §2.2 με padding και pooling: padding=2 σε κάθε convolution και MaxPool2d(kernel_size=2) μετά από κάθε conv, καμία activation ακόμα.
- §2.3 με activations: προσθέτετε ReLU μετά από κάθε convolution (πριν το pooling) και πριν από κάθε γραμμικό επίπεδο του FC head, εκτός από το τελευταίο που οδηγεί στο output.

Εκπαιδεύστε κάθε έκδοση με τα settings του Ερωτήματος 1 (Adam, lr=0.001, 30 εποχές) και μετρήστε τον συνολικό αριθμό παραμέτρων ανά έκδοση (`sum(p.numel() for p in model.parameters())`). Εκπαιδεύστε την §2.3 και σε CPU και σε GPU.

Q2.1 (4 μον.)

Συμπληρώστε τον πίνακα:

Έκδοση	# params	Val Acc	Test Acc	F1 macro
§2.1 (χωρίς pad/pool/activ)				
§2.2 (με pad και pool)				

§2.3 (με ReLU)				
----------------	--	--	--	--

Ποια αλλαγή είχε τη μεγαλύτερη επίδραση στην επίδοση και ποια στον αριθμό παραμέτρων; Αν η §2.1 πετυχαίνει χαμηλή ή σχεδόν τυχαία απόδοση, εξηγήστε τεχνικά γιατί συμβαίνει αυτό, πέρα από τη γενική παρατήρηση ότι λείπουν τα activations.

Q2.2 (3 μον.)

Συγκρίνετε το τελικό CNN (§2.3) με το FFN του Ερωτήματος 1 σε δύο μετρικές: τον αριθμό των παραμέτρων και το test F1 macro. Το CNN έχει περισσότερες ή λιγότερες παραμέτρους από το FFN; Πόσο διαφορετική είναι η επίδοση; Συζητήστε γιατί ο αριθμός των παραμέτρων από μόνος του δεν προβλέπει την επίδοση, και ποιο ρόλο παίζει εδώ η δομή των παραμέτρων (πώς οργανώνονται και ποια prior knowledge κωδικοποιούν για εικόνες). Συγκρίνετε επίσης τα confusion matrices: σε ποια κινήματα βελτιώνεται ουσιαστικά το CNN; Πώς συγκρίνεται το CPU-vs-GPU speedup του CNN με αυτό του FFN, και γιατί;

Q2.3 (3 μον.)

Σενάριο: Ένα μουσείο σχεδιάζει σύστημα αυτόματης υφολογικής ταξινόμησης για ψηφιοποιημένη συλλογή 2.000.000 έργων. Σας προτείνεται να βαθύνετε το δίκτυο (δέκα συνελκτικά επίπεδα αντί για τέσσερα), με αναμενόμενο όφελος +2% στο test accuracy αλλά πολλαπλάσιο computational cost. Βασιστείτε στους χρόνους CPU/GPU που μετρήσατε (§1 και §2.3) και στο train-val behavior που παρατηρήσατε. Σε ποιες περιπτώσεις είναι λογική αυτή η επιλογή και σε ποιες όχι; Σκεφτείτε deployment context (real-time έναντι batch processing), κόστος ενέργειας, και ανάγκη ερμηνευσιμότητας από επιμελητές.

3. Βελτίωση Επίδοσης

Σε αυτή την ενότητα θα εφαρμόσετε σύγχρονες τεχνικές βελτίωσης CNN: reproducibility, εναλλακτικοί optimizers, learning rate scheduling, batch normalization, data augmentation, και regularization. Κάθε βήμα ξεκινάει από την αρχιτεκτονική της §2.3.

3.1 Reproducibility, optimizers, LR schedule, BatchNorm

Για να συγκρίνετε αξιόπιστα τις τεχνικές που ακολουθούν, η εκπαίδευση πρέπει να είναι πλήρως αναπαραγώγιμη. Σε διαφορετική περίπτωση, τυχαίες παραλλαγές στην αρχικοποίηση των βαρών ή στη σειρά των batches μπορεί να μιμηθούν τα οφέλη μιας τεχνικής.

Συμβουλευτείτε το [PyTorch documentation για reproducibility](#) και εφαρμόστε τα κατάλληλα settings: seed για random, numpy, torch, και torch.cuda.manual_seed_all, καθώς και torch.backends.cudnn.deterministic = True και torch.backends.cudnn.benchmark = False. Εκτελέστε δύο φορές την εκπαίδευση της §2.3 και επιβεβαιώστε ότι παίρνετε ακριβώς το ίδιο train loss σε κάθε εποχή και τις ίδιες επιδόσεις στο test set.

Στη συνέχεια εκπαιδεύστε την §2.3 με πέντε διαφορετικούς optimizers ώστε να δείτε αν κάποιος ξεπερνά τον Adam (που είναι ήδη το baseline σας από το Ερώτημα 2). Συμβουλευτείτε το [PyTorch optimizers documentation](#):

- SGD με lr=0.01
- SGD με momentum 0.9 και lr=0.01
- Adam με lr=0.001 (baseline)
- RMSprop με lr=0.001
- AdamW με lr=0.001 και weight_decay=0.01

Στον καλύτερο optimizer που προέκυψε από τη σύγκριση, προσθέστε CosineAnnealingLR ως learning rate scheduler με T_max=30 (ίσο με τον αριθμό εποχών), και συγκρίνετε με την εκδοχή χωρίς scheduler. Τέλος, εισάγετε nn.BatchNorm2d πριν από κάθε ReLU σε όλα τα convolutional επίπεδα, και εκπαιδεύστε με τον βέλτιστο optimizer και τον scheduler.

Q3.1 (6 μον.)

Συμπληρώστε τον πίνακα optimizers (χωρίς LR schedule και χωρίς BatchNorm):

Optimizer	Val Acc	Test Acc	F1 macro	Best epoch
SGD				
SGD με momentum				
Adam				
RMSprop				
AdamW				

Ποιος optimizer επικράτησε σε επίδοση και σε ταχύτητα σύγκλισης (best epoch); Συζητήστε την παρατηρούμενη διαφορά μεταξύ SGD και adaptive optimizers (Adam, RMSprop, AdamW). Πώς επηρεάστηκε η εκπαίδευση από το cosine LR schedule και από την προσθήκη του BatchNorm; Διατυπώστε διαισθητικά τι κάνει ένα LR schedule κατά τη διάρκεια της εκπαίδευσης.

3.2 Data augmentation και regularization

Στην §3.1 πιθανώς παρατηρήσατε σημαντική απόκλιση μεταξύ train και validation loss, που υποδηλώνει overfitting. Σε αυτή την υποενότητα θα προσπαθήσετε να το μειώσετε με δύο συμπληρωματικούς τρόπους: από τη μεριά των δεδομένων με augmentation, και από τη μεριά των παραμέτρων με regularization.

Η baseline αρχιτεκτονική είναι το τελικό CNN της §3.1 (§2.3 με BatchNorm, τον καλύτερο optimizer, και cosine LR schedule). Θα την χρησιμοποιήσετε σε όλα τα πειράματα που ακολουθούν, εκπαιδευμένη σε 60 εποχές (αντί 30), προσαρμόζοντας αντίστοιχα και το T_max του scheduler.

Για το data augmentation εφαρμόστε σε κάθε training image (όχι σε val/test) μέσω torchvision.transforms:

- RandomHorizontalFlip(p=0.5)
- RandomCrop(32, padding=4)
- ColorJitter(brightness=0.1, contrast=0.1)

Ορισμένοι μετασχηματισμοί (transforms) που εφαρμόζονται συχνά σε φυσικές εικόνες (π.χ. RandomRotation, RandomVerticalFlip, ή ακραίο ColorJitter) είναι σημασιολογικά λανθασμένες για πίνακες ζωγραφικής. Σκεφτείτε γιατί, πριν εφαρμόσετε οποιαδήποτε από αυτές.

Για το regularization εφαρμόστε weight decay=1e-4 στον optimizer και Dropout (p=0.3) πριν από κάθε γραμμικό επίπεδο του FC head.

Δοκιμάστε τέσσερις ρυθμίσεις: baseline (καμία αύξηση επιπλέον), μόνο augmentation, μόνο regularization, και τέλος και τα δύο μαζί. Για κάθε ρύθμιση, σχεδιάστε τα train και validation loss curves στο ίδιο διάγραμμα.

Q3.2 (6 μον.)

Συμπληρώστε τον πίνακα:

Ρύθμιση	Best epoch	Val F1	Test F1	Train-Val gap
Καμία (§3.1 baseline, 60 εποχές)				
Με data augmentation				
Με weight decay και dropout				
Με augmentation και regularization				

Σε ποια ρύθμιση μειώνεται περισσότερο το train-val gap (val_loss - train_loss), και σε ποια ανεβαίνει η επίδοση στο test set; Συμπίπτουν αυτά τα δύο; Τι συμπεραίνετε για τη σχέση μεταξύ καλής γενίκευσης και απουσίας overfitting; Ποιους μετασχηματισμούς αποφασίσατε ότι θα ήταν λάθος για πίνακες ζωγραφικής, και γιατί;

4. Transfer Learning με Προεκπαιδευμένο ResNet18

Έχετε φτάσει στα όρια της απόδοσης που μπορείτε να πετύχετε εκπαιδώντας από το μηδέν σε 3.200 πίνακες. Σε αυτή την ενότητα θα χρησιμοποιήσετε ένα προεκπαιδευμένο ResNet18, ένα δίκτυο 18 επιπέδων με περίπου 11 εκατομμύρια παραμέτρους, εκπαιδευμένο σε 1,2 εκατομμύρια εικόνες του ImageNet (γάτες, σκύλοι, αυτοκίνητα, αντικείμενα της καθημερινής ζωής, όχι όμως πίνακες). Θα ελέγξετε αν αυτά που έμαθε σε τόσο διαφορετικό domain μεταφέρονται στην ταξινόμηση τέχνης.

4.1 Linear probe σε frozen ResNet18

Φορτώστε το προεκπαιδευμένο μοντέλο και τροποποιήστε το ως εξής:

- Φορτώστε `torchvision.models.resnet18(weights="IMAGENET1K_V1")`.
- Παγώστε όλα τα βάρη του ώστε να μην ενημερώνονται κατά την εκπαίδευση (`for p in model.parameters(): p.requires_grad = False`).
- Αντικαταστήστε το τελευταίο fully-connected επίπεδο (`model.fc`, με 1.000 outputs για τις ImageNet κλάσεις) με ένα νέο `nn.Linear(512, 4)` που θα εκπαιδευτεί από το μηδέν.
- Κάντε `resize` τις εικόνες στο 224×224 (το input size που περιμένει το ResNet18). Μπορείτε να `upscale` τα 32×32 tensors με `torch.nn.functional.interpolate(images, size=224, mode='bilinear', align_corners=False)`.
- Κανονικοποιήστε τις εικόνες με τα ImageNet statistics: `mean=[0.485, 0.456, 0.406]`, `std=[0.229, 0.224, 0.225]`.

Εκπαιδεύστε μόνο το νέο `fc` layer με Adam (`lr=0.001`) για 10 εποχές, και αξιολογήστε στο test set.

Σημείωση: το ResNet18 έχει εκπαιδευτεί σε εικόνες 224×224. Το `upsampling` από 32×32 εισάγει artifacts που μπορεί να επηρεάσουν την απόδοση.

Q4.1 (4 μον.)

Συμπληρώστε τον τελικό πίνακα σύγκρισης:

Μοντέλο	Test Acc	F1 macro	Trainable params	Training time
FFN flat pixels (§1)				
CNN βέλτιστο (§3.2)				
Frozen ResNet18 με linear probe (§4.1)				

Συγκρίνετε την επίδοση του frozen ResNet18 με το CNN που εκπαιδεύσατε εσείς από το μηδέν. Αν το ResNet18 αποδίδει καλύτερα, παρότι δεν έχει δει ποτέ έργο τέχνης κατά την προεκπαίδευσή του, πώς εξηγείται αυτό και τι σας λέει για το πώς ένα δίκτυο μαθαίνει από το ImageNet αναπαραστάσεις γενικά χρήσιμες για όραση, ακόμα και σε εντελώς διαφορετικό domain όπως η τέχνη; Αν αποδίδει χειρότερα, ποιοι παράγοντες της συγκεκριμένης υλοποίησης (π.χ. η αρχική ανάλυση 32×32 και το `upsampling` σε 224×224, ή ο περιορισμός σε linear probe) μπορεί να ευθύνονται;

5. Τι Έμαθαν τα Δίκτυα

Έχετε τώρα δύο εκπαιδευμένα δίκτυα: το CNN που φτιάξατε εσείς (§3.2) και το frozen ResNet18 (§4.1). Σε αυτή την ενότητα θα συγκρίνετε τι έχει μάθει το καθένα στο πρώτο συνελικτικό του επίπεδο.

5.1 Σύγκριση πρώτων conv layers

Εξάγετε τα βάρη του πρώτου convolutional layer από κάθε δίκτυο και οπτικοποιήστε τα ως μικρές RGB εικόνες. Από το δικό σας CNN, το tensor βαρών έχει σχήμα [16, 3, 5, 5]: 16 φίλτρα, καθένα 5×5 σε 3 κανάλια. Κανονικοποιήστε κάθε φίλτρο στο [0, 1] (min-max ανά φίλτρο) και οπτικοποιήστε τα 16 φίλτρα σε διάταξη 4×4. Από το ResNet18, το tensor `model.conv1.weight` έχει σχήμα [64, 3, 7, 7]. επιλέξτε τα 16 πρώτα φίλτρα και οπτικοποιήστε με τον ίδιο τρόπο.

Στη συνέχεια εξάγετε τα activation maps του πρώτου conv layer για ένα τυχαίο δείγμα κάθε κλάσης. Επιλέξτε έναν πίνακα από κάθε κλάση από το test set, κάντε forward pass μέσω κάθε δικτύου, και αποθηκεύστε τα activations του πρώτου conv layer μετά τη ReLU. Οπτικοποιήστε τα 16 πρώτα channels ως heatmaps για κάθε πίνακα και κάθε δίκτυο.

Q5.1 (3 μον.)

Συγκρίνετε τα 16 πρώτα φίλτρα του δικού σας CNN με αυτά του ResNet18. Σε ποιο δίκτυο τα φίλτρα φαίνονται πιο καθαρά και ερμηνεύσιμα (αναγνωρίσιμα edge detectors, color blobs, αντιθέσεις φωτεινού-σκούρου); Είναι αυτό αναμενόμενο, και γιατί; Από τα activation maps, ποια φίλτρα ενεργοποιούνται περισσότερο για κάθε κίνηση, και υπάρχει κάποιο που ξεχωρίζει εμφανώς (π.χ. ένα φίλτρο που ανάβει σε σκούρες περιοχές για το Μπαρόκ); Συνδέστε τις παρατηρήσεις σας με όσα γράψατε στο Q0.1: τι από αυτά που εσείς προβλέψατε ως διαφοροποιητικό μεταξύ κινημάτων φαίνεται να έχει πιάσει κάθε δίκτυο; Τέλος, αντιπαραβάλετε τα συνελικτικά φίλτρα και των δύο δικτύων με τις εικόνες βαρών του πρώτου επιπέδου του FFN που οπτικοποιήσατε στο Q1.2: τι παρατηρείτε όταν περνάμε από flat-pixel βάρη σε χωρικά συνελικτικά φίλτρα;

Προφορική Εξέταση (60 μονάδες)

Διάρκεια: 15 λεπτά ανά φοιτητή/τρια. Θα ερωτηθείτε πάνω στο notebook και τα αποτελέσματά σας, και θα ζητηθεί να εξηγήσετε τις επιλογές σας και να απαντήσετε σε νέα ερωτήματα.

Η προφορική εξέταση βασίζεται στο notebook όπως έχει υποβληθεί. Αν τα αποτελέσματα (μοντέλα, διαγράμματα, metrics) δεν είναι διαθέσιμα ή δεν μπορούν να παρουσιαστούν άμεσα, η εργασία θεωρείται μη επαρκώς τεκμηριωμένη.

Κλίμακα βαθμολόγησης προφορικής εξέτασης: 55–60 κατανόηση σε βάθος και ικανότητα απάντησης σε νέα ερωτήματα, 40–54 επαρκής κατανόηση και αιτιολόγηση βασικών επιλογών, 20–39 επιφανειακή γνώση και δυσκολία στα follow-up ερωτήματα, 0–19 αδυναμία εξήγησης.

Βαθμολογία

Ενότητα	Μονάδες
---------	---------

0. Δεδομένα	3
1. Feedforward Neural Network σε ακατέργαστα pixels	8
2. Convolutional Neural Network	10
3. Βελτίωση Επίδοσης	12
4. Transfer Learning με Προεκπαιδευμένο ResNet18	4
5. Τι Έμαθαν τα Δίκτυα	3
Σύνολο Notebook	40
Προφορική Εξέταση	60
Σύνολο	100

Σε περίπτωση τεχνικών προβλημάτων

Αν αντιμετωπίσετε σοβαρό τεχνικό πρόβλημα (CUDA out-of-memory, μη σύγκλιση μοντέλου, ασταθή αποτελέσματα μεταξύ runs):

- Γράψτε στο notebook τι δοκιμάσατε, τι error πήρατε, και τι κάνατε για να το λύσετε.
- Αν ένα μοντέλο παρουσιάζει σφάλμα ή δεν εκπαιδεύεται, εξηγήστε τι νομίζετε ότι φταίει και τι θα περιμένατε αν δούλενε.
- Τεχνικό πρόβλημα που τεκμηριώνεται σωστά δεν συνεπάγεται μείωση βαθμολογίας, αρκεί η υπόλοιπη εργασία να είναι πλήρης.

Χρήση Εργαλείων AI

Μπορείτε να χρησιμοποιήσετε εργαλεία AI (ChatGPT, Claude, Copilot κ.ά.) για debugging, ερωτήματα σύνταξης PyTorch, αποσαφήνιση εννοιών. Δεν επιτρέπεται να ζητήσετε από AI να γράψει τις απαντήσεις στα ερωτήματα ή να ερμηνεύσει αποτελέσματα για λογαριασμό σας. Στην προφορική εξέταση ενδέχεται να σας ζητηθεί να εξηγήσετε τμήματα κώδικα που φαίνεται να έχουν παραχθεί από εργαλείο AI.

Υποχρεωτικά συμπεριλάβετε αρχείο `AI_declaration.md` στο οποίο αναφέρετε ποια εργαλεία χρησιμοποιήσατε, για ποιον σκοπό, και δηλώνετε ότι κατανοείτε τον κώδικα και τα αποτελέσματα.

Οδηγίες υλοποίησης και υποβολής

Η εργασία υλοποιείται σε Jupyter notebook (Google Colab ή αντίστοιχο περιβάλλον). Βεβαιωθείτε ότι το notebook εκτελείται χωρίς σφάλματα από την αρχή μέχρι το τέλος (Run All) πριν την υποβολή. Κάθε ερώτημα απαντάται σε ένα κελί markdown αμέσως

μετά τον αντίστοιχο κώδικα, με σαφή μορφοποίηση και δομή ώστε να διευκολύνεται η ανάγνωση και η αξιολόγηση.

Για αναπαραγωγικότητα, προσθέστε στην αρχή του notebook:

```
import numpy as np, random, torch
SEED = 42
np.random.seed(SEED)
random.seed(SEED)
torch.manual_seed(SEED)
torch.cuda.manual_seed_all(SEED)
torch.backends.cudnn.deterministic = True
torch.backends.cudnn.benchmark = False
```

Σε όλα τα `train_test_split` και `sklearn/PyTorch initializations` που εμπλέκουν τυχαιότητα, χρησιμοποιήστε `random_state=42` ή ισοδύναμο.

Αρχείο ZIP με ονομασία `AM_Επώνυμο.zip` (π.χ. `1115XXXX_Παπαδόπουλος.zip`), που περιέχει:

- `AM_Επώνυμο.ipynb`: notebook με κώδικα, αποτελέσματα, και απαντήσεις.
- `README.txt`: ονοματεπώνυμο, AM, λίστα εξαρτήσεων (`pip install ...`), εκτιμώμενος χρόνος εκτέλεσης σε GPU.
- `AI_declaration.md`.

Μην συμπεριλάβετε το αρχείο `wikiart_hw2.npz` στο ZIP. Υποθέτουμε ότι ο διορθωτής το έχει τοπικά. Στον κώδικά σας θεωρήστε ότι βρίσκεται στον ίδιο φάκελο με το notebook.